# *Android*
# Unidad 2. Diseño de Interfaces de Usuario

PROGRAMACIÓN MULTIMEDIA Y DE DISPOSITIVOS MÓVILES

2º DAM

# Índice


1. *Jeckpack Compose*
2. Elementos de texto
3. Imágenes
4. Componente *Scaffold*
5. *Layouts*
6. Botones
7. *Mockups*
8. Localización

# 1. *Jetpack Compose*

*Android Studio* nos permite realizar el diseño de las interfaces de usuario utilizando el kit de herramientas *Jetpack Compose* siendo *Jetpack* un conjunto de librerías de desarrollo moderno de aplicaciones en Android.

Las características de *Jetpack Compose* son:

- Es una forma moderna de crear aplicaciones nativas en *Android*.
- Simplifica y acelera el desarrollo de la IU en *Android*.
- Ha sido diseñado y desarrollado por el equipo de *Android*.
- Es de código abierto y colabora activamente con toda la comunidad de *Android*.
- Utiliza el lenguaje de programación *Kotlin* (también lo utilizaremos para programar la funcionalidad de la aplicación).

# 1. *Jetpack Compose*

*Compose* utiliza funciones de componibilidad que son los componentes básicos para crear IU.

Las funciones de componibilidad:

- Tienen la anotación ***@Composable***.
- No devuelven ningún parámetro.
- Describen cómo debe de visualizarse esa parte de IU.

```
32  @Composable
33  fun Greeting(name: String, modifier: Modifier = Modifier) {
34      Text(
35          text = "Hello $name!",
36          modifier = modifier
37      )
38  }
```

# 1. *Jetpack Compose*: Anotaciones

Las anotaciones añaden información extra al código que ayuda al compilador o a desarrolladores a entender el código de la aplicación.

Para añadir una anotación se debe escribir @ seguido de la palabra reservada de la anotación.

Las anotaciones permiten pasar varios parámetros de entrada.

Ejemplos:

- *@Composable*
- *@Preview(showBackground = true)*
- *@Field*
- *@Entity*

# 1. *Jetpack Compose*: Funciones de componibilidad

Una función de componibilidad no devuelve ningún valor y tiene la anotación *@Composable*. Esta anotación informa al compilador de *Compose* que la función está diseñada para trabajar con la IU.

El nombre de la función debe **comenzar por mayúscula** y debe ser un **sustantivo**, no un verbo, adjetivo o adverbio.

Nombres no válidos serían:

- *DibujarPerfilUsuario*
- *Brillante*
- *Fuera*

MainActivity.kt

```kotlin
package com.example.myapplication

import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    MyApplicationTheme {
        Greeting( name: "Android")
    }
}
```

El bloque *setContent* define el diseño de la actividad. Aquí se llaman a las funciones que admiten composición.

*MyApplicationTheme* indica que se va a utilizar el tema creado para la aplicación. Lo podemos encontrar dentro del fichero *ui.theme/Theme.kt*.

*Scaffold* es un contenedor donde le aplicamos un modificador.
A él le añadimos la función de composición *Greeting* pasándole el texto “Android” y el modificador heredado.

Un modificador indica a un elemento de la IU cómo aparecer o comportarse dentro del contenedor.

MainActivity.kt

```kotlin
package com.example.myapplication

import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    MyApplicationTheme {
        Greeting( name: "Android")
    }
}
```

Función componible a la que le pasamos un *String* para el nombre a escribir y un modificador predeterminado.

El elemento de texto *Text* rellena su atributo *text* usando el *String* pasado por parámetro y asigna a su atributo *modifier* el modificador también pasado por parámetro.

MainActivity.kt

```kotlin
package com.example.myapplication

import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    MyApplicationTheme {
        Greeting( name: "Android")
    }
}
```

Función componible que realiza la vista previa de la función *Greeting*.

También hace uso del tema *MyApplicationTheme*.

# 2. Elementos de texto

*Compose* ofrece dos elementos básicos de texto: *Text* y *TextField*.

Ambos admiten composición y se basan en *Material Design*.

El uso básico para mostrar un texto es utilizar la función *Text* pasándole un *String* como argumento:

```
49  @Composable
50  fun TextoBasico() {
51      Text( text: "Ejemplo")
52  }
```

Aunque esto no es lo recomendable ya que al pasarle un *String* directamente no podremos localizar nuestra aplicación en diferentes idiomas. Además, es importante pasarle el modificador.

# 2. Elementos de texto: *Text*

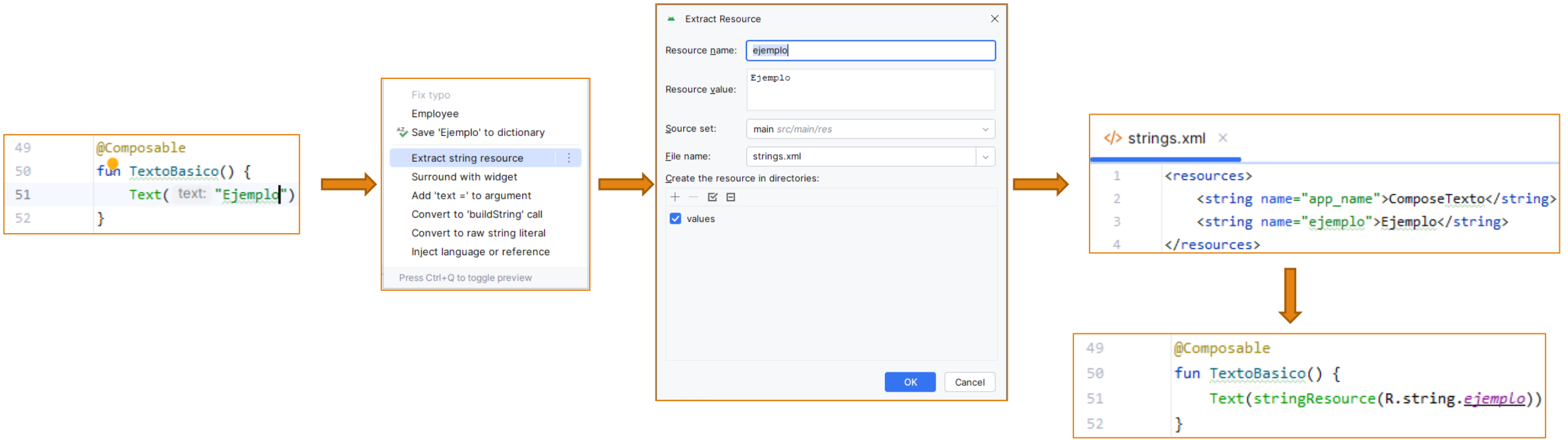
Para localizar la aplicación, los *Strings* utilizados en la IU deben estar almacenados en el fichero *strings.xml* que, como veremos más adelante, habrá uno asociado a cada idioma utilizado.

Para recuperar los *Strings* y poder utilizarlos en las funciones, tendremos que hacer uso de la función *stringResource* a la que le pasaremos como parámetro el *String* guardado en el fichero *strings.xml*.

```
49      @Composable
50      fun TextoBasico() {
51          Text(stringResource(R.string.ejemplo))
52      }
```

# 2. Elementos de texto: *Text*

La forma rápida para extraer el texto, es clicar sobre el *String* a extraer, pulsar sobre la bombilla, elegir la opción ***Extract string resource***, modificar el nombre del recurso, si queremos, y clicar en OK. Esto añadirá el *String* al fichero *strings.xml* y ya podremos utilizar el recurso.



# 2. Elementos de texto: *Text* Estilos

Los estilos de texto más comunes son:

- Color del texto
- Tamaño del texto
- Cursiva y negrita

Los estilos de párrafo más comunes son:

- Alineación del texto
- Altura de la línea de texto

# 2. Elementos de texto: *Text* Estilos

**Color del texto**

Para cambiar el color del texto se utiliza el argumento *color* de la función *Text*.

Existen varias formas de modificar el color de un texto:

- Utilizando la clase ***Color*** con los colores predefinidos o indicando el código del color a usar.

```
34      @Composable
35      fun EstiloTexto(texto: String, modifier: Modifier = Modifier) {
36          Text(
37              text = texto,
38              color = Color.Blue
39          )
40      }
```

```
34      @Composable
35      fun EstiloTexto(texto: String, modifier: Modifier = Modifier) {
36          Text(
37              text = texto,
38              color = Color( color: 0xFFD0BCFF)
39          )
40      }
```

# 2. Elementos de texto: *Text* Estilos

**Color del texto**

Existen varias formas de modificar el color de un texto:

- Accediendo a los colores que tenemos almacenados en el fichero ***colors.xml***

```
35  @Composable
36  fun EstiloTexto(texto: String, modifier: Modifier = Modifier) {
37      Text(
38          text = texto,
39          color = colorResource(id = R.color.purple_200)
40      )
41  }
```

colors.xml

```
1   <?xml version="1.0" encoding="utf-8"?>
2   <resources>
3       <color name="purple_200">#FFBB86FC</color>
4       <color name="purple_500">#FF6200EE</color>
5       <color name="purple_700">#FF3700B3</color>
6       <color name="teal_200">#FF03DAC5</color>
7       <color name="teal_700">#FF018786</color>
8       <color name="black">#FF000000</color>
9       <color name="white">#FFFFFFFF</color>
10  </resources>
```

# 2. Elementos de texto: *Text* Estilos

**Color del texto**

Existen varias formas de modificar el color de un texto:

- Utilizando los colores definidos en la clase ***ui.theme/Color.kt*** de nuestra aplicación

```
14  import com.example.composetexto.ui.theme.Purple80
```

```
33  @Composable
34  fun EstiloTexto(texto: String, modifier: Modifier = Modifier) {
35      Text(
36          text = texto,
37          color = Purple80
38      )
39  }
```

Color.kt

```
1   package com.example.composetexto.ui.theme
2
3   import androidx.compose.ui.graphics.Color
4
5   val Purple80 = Color( color: 0xFFD0BCFF)
6   val PurpleGrey80 = Color( color: 0xFFCCC2DC)
7   val Pink80 = Color( color: 0xFFEFB8C8)
8
9   val Purple40 = Color( color: 0xFF6650a4)
10  val PurpleGrey40 = Color( color: 0xFF625b71)
11  val Pink40 = Color( color: 0xFF7D5260)
```

# 2. Elementos de texto: *Text* Estilos

**Color del texto**

Existen varias formas de modificar el color de un texto:

- Utilizando los colores definidos en el tema de la aplicación, *Theme.kt* (Recomendable)

```
33  @Composable
34  fun EstiloTexto(texto: String, modifier: Modifier = Modifier) {
35      Text(
36          text = texto,
37          color = MaterialTheme.colorScheme.primary
38      )
39  }
```

```
14  private val DarkColorScheme = darkColorScheme(
15      primary = Purple80,
16      secondary = PurpleGrey80,
17      tertiary = Pink80
18  )
19
20  private val LightColorScheme = lightColorScheme(
21      primary = Purple40,
22      secondary = PurpleGrey40,
23      tertiary = Pink40
```

# 2. Elementos de texto: *Text* Estilos

**Tamaño del texto**

Para cambiar el tamaño del texto se utiliza el argumento *fontSize* de la función *Text*.

El tamaño se asigna utilizando la unidad de medida ***SP*** (*Scalable Pixels*) y a través de la propiedad de extensión **.sp**. La podremos utilizar tanto para *Int* como para *Float* y *Double*.

Las funciones y propiedades de extensión nos permiten extender de una clase con sus funciones y propiedades sin necesidad de heredar de ella.

```
13      import androidx.compose.ui.unit.sp
```

```
33      @Composable
34      fun EstiloTexto(texto: String, modifier: Modifier = Modifier) {
35          Text(
36              text = texto,
37              fontSize = 50.sp
38          )
39      }
```

# 2. Elementos de texto: *Text* Estilos

**Cursiva y negrita**

Para poner el texto en cursiva y en negrita se utilizan los argumentos *fontStyle* y *fontWeight* respectivamente de la función *Text*.

```
12  import androidx.compose.ui.text.font.FontStyle
```

```
33  @Composable
34  fun EstiloTexto(texto: String, modifier: Modifier = Modifier) {
35      Text(
36          text = texto,
37          fontStyle = FontStyle.Italic
38      )
39  }
```

```
12  import androidx.compose.ui.text.font.FontWeight
```

```
33  @Composable
34  fun EstiloTexto(texto: String, modifier: Modifier = Modifier) {
35      Text(
36          text = texto,
37          fontWeight = FontWeight.Bold
38      )
39  }
```

# 2. Elementos de texto: *Text* Estilos

**Alineación del texto**

Para alinear el texto horizontalmente se utiliza el argumento *textAlign* de la función *Text*.

Por defecto, la función *Text* alinea el texto según el alfabeto utilizado:

- Borde izquierdo para alfabetos de izquierda a derecha, como el latino.
- Borde derecho para alfabetos de derecha a izquierda, como el árabe.

```
12  import androidx.compose.ui.text.style.TextAlign
```

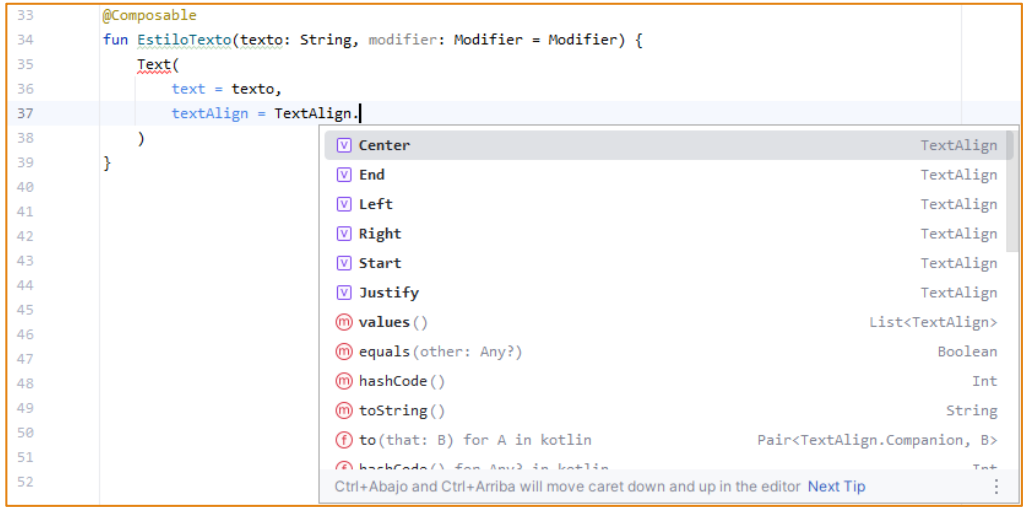
```
33  @Composable
34  fun EstiloTexto(texto: String, modifier: Modifier = Modifier) {
35      Text(
36          text = texto,
37          textAlign = TextAlign.Center
38      )
39  }
```

# 2. Elementos de texto: Estilos

## Alineación del texto

Si echamos un vistazo a las posibilidades que nos ofrece *textAlign* para alinear el texto podemos observar que aparecen ***Start*** y ***End***. ***Start*** hace referencia a la izquierda y ***End*** a la derecha.

Es importante saber que es preferible usar ***Start*** y ***End*** en lugar de ***Left*** y ***Right***, ya que alinean el texto según la orientación de texto del idioma preferido. Por ejemplo, ***End*** se alinea con el lado derecho para el texto en inglés y con el lado izquierdo para el texto en árabe, pero ***Right*** se alinea con el lado derecho, sin importar qué alfabeto se use.



# 2. Elementos de texto: *Text* Estilos

**Altura de la línia de texto**

Para configurar la altura de la línea de texto se utiliza el argumento *lineHeight* de la función *Text*.

Al igual que en el tamaño de texto también utilizamos la unidad de medida ***sp***, aunque es posible utilizar la unidad de texto ***em*** (tamaño de fuente relativo) para mejorar la precisión.

```
34      @Composable
35      fun EstiloTexto(texto: String, modifier: Modifier = Modifier) {
36          Text(
37              text = texto,
38              lineHeight = 100.sp
39          )
40      }
```

# 2. Elementos de texto: *TextField*

La función *TextField* nos permite que el usuario inserte y modifique texto siguiendo la implementación de *Material Design*.

El estilo por defecto es el relleno. Para la versión del estilo con contorno tendremos que utilizar la función *OutlinedTextField*.

# 2. Elementos de texto: *TextField*

Los parámetros básicos que acepta la función *TextField* son *value*, *onValueChange* y *label*:

- *value* mostrará el valor inicial del *TextField*.
- *onValueChange* recogerá el texto escrito por el usuario.
- *label* mostrará un texto a modo de ayuda.

```
13  import androidx.compose.runtime.getValue
14  import androidx.compose.runtime.mutableStateOf
15  import androidx.compose.runtime.remember
16  import androidx.compose.runtime.setValue
```

```
39  @Composable
40  fun EjemploTextFieldRelleno() {
41      var texto by remember { mutableStateOf( value: "Texto") }
42
43      TextField(
44          value = texto,
45          onValueChange = { texto = it },
46          label = { Text( text: "Etiqueta") }
47      )
48  }
```



# 2. Elementos de texto: *TextField*

Otros parámetros útiles son *singleLine*, *maxLines*, *textStyle* o *keyboardOptions*:

- *singleLine* indicará que el *TextField* es de una sola línea.
- *maxLines* indicará el número de líneas que puede tener el *TextField*.
- *textStyle* añadirá un estilo.
- *keyboardOptions* nos permitirá elegir el tipo de teclado que se usará para insertar texto.

```
44      @Composable
45      fun EjemploTextFieldRelleno() {
46          var texto by remember { mutableStateOf( value: "0") }
47
48          TextField(
49              value = texto,
50              onValueChange = { texto = it },
51              label = { Text( text: "Etiqueta") },
52              singleLine = true,
53              keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
54              textStyle = TextStyle(color = Color.Red, fontWeight = FontWeight.Bold)
55          )
56      }
```



# 2. Elementos de texto: *TextField*

El uso de la función *OutlinedTextField* es exactamente igual al anterior.

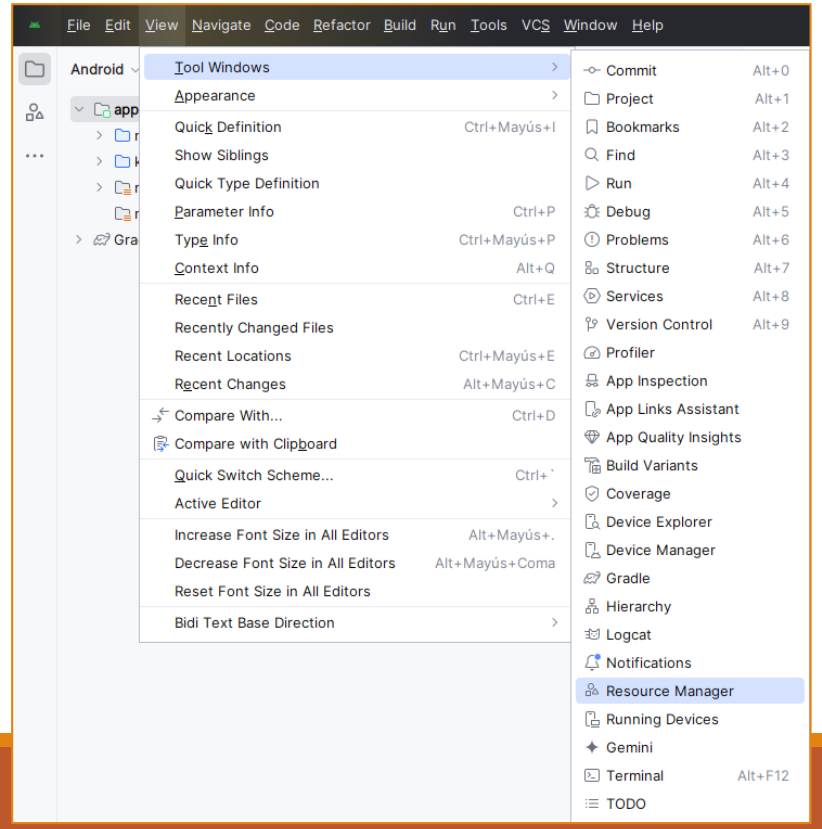
```
45      @Composable
46      fun EjemploTextFieldRelleno() {
47          var texto by remember { mutableStateOf( value: "0") }
48
49          OutlinedTextField(
50              value = texto,
51              onValueChange = { texto = it },
52              label = { Text( text: "Etiqueta") },
53              singleLine = true,
54              keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
55              textStyle = TextStyle(color = Color.Red, fontWeight = FontWeight.Bold)
56          )
57      }
```

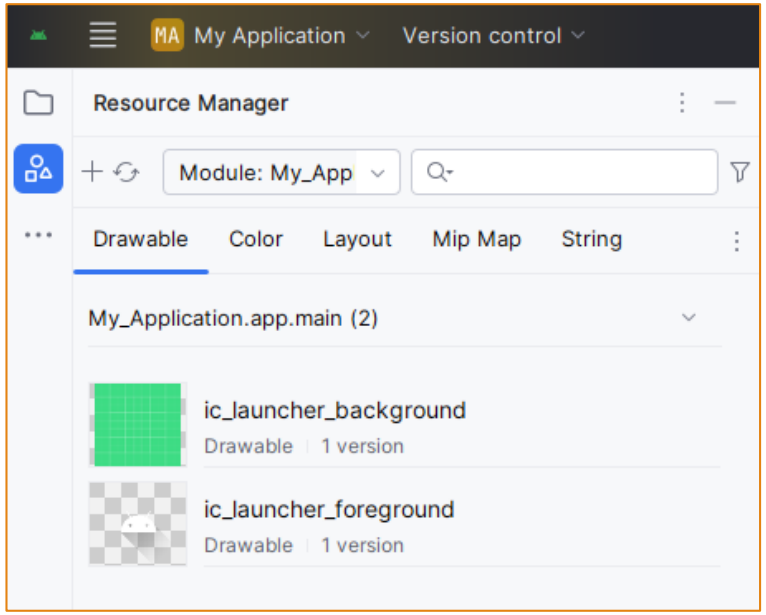


# 3. Imágenes: Importar al proyecto

Para añadir una imagen a nuestro proyecto tenemos dos opciones, clicar en:

- ***View > Tool Windows > Resource Manager*** o
- En la pestaña ***Resource Manager*** debajo de la pestaña ***Project***.

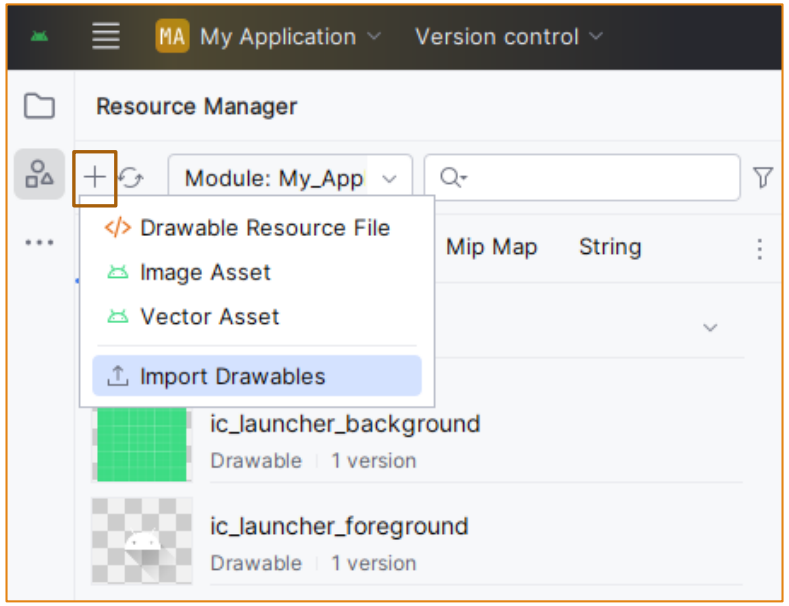


# 3. Imágenes: Importar al proyecto

***Resource Manager*** es una ventana de herramientas que te permite importar, crear, administrar y usar recursos en tu proyecto.

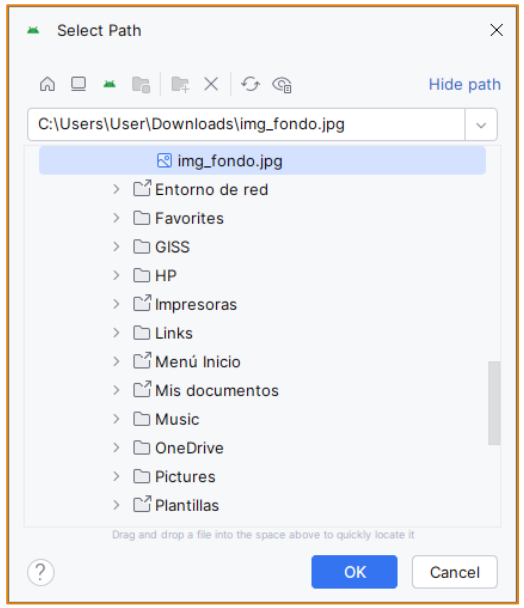
Para importar una imagen haremos clic en ***+ > Import Drawables***.

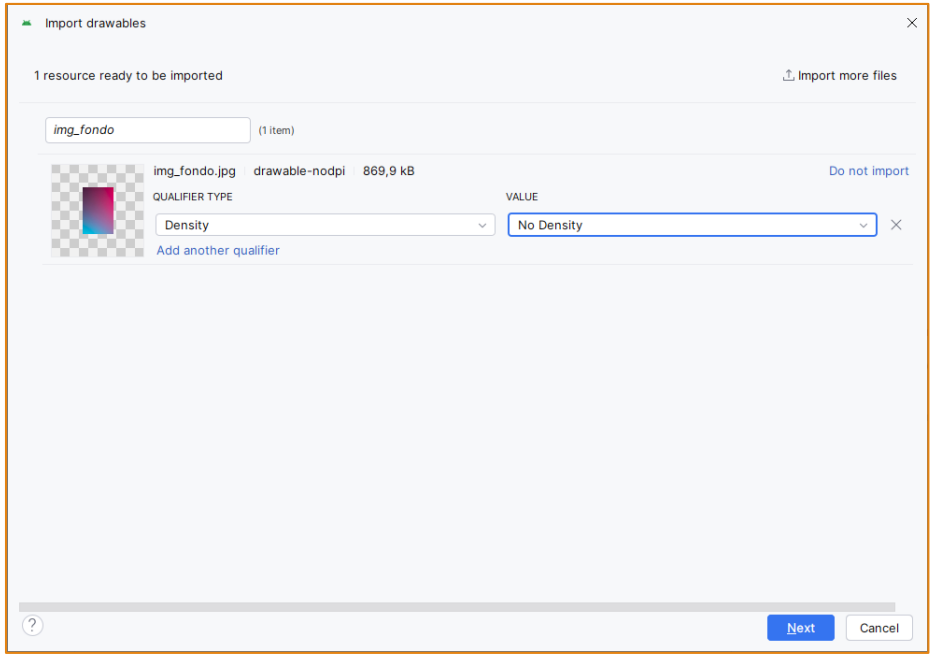


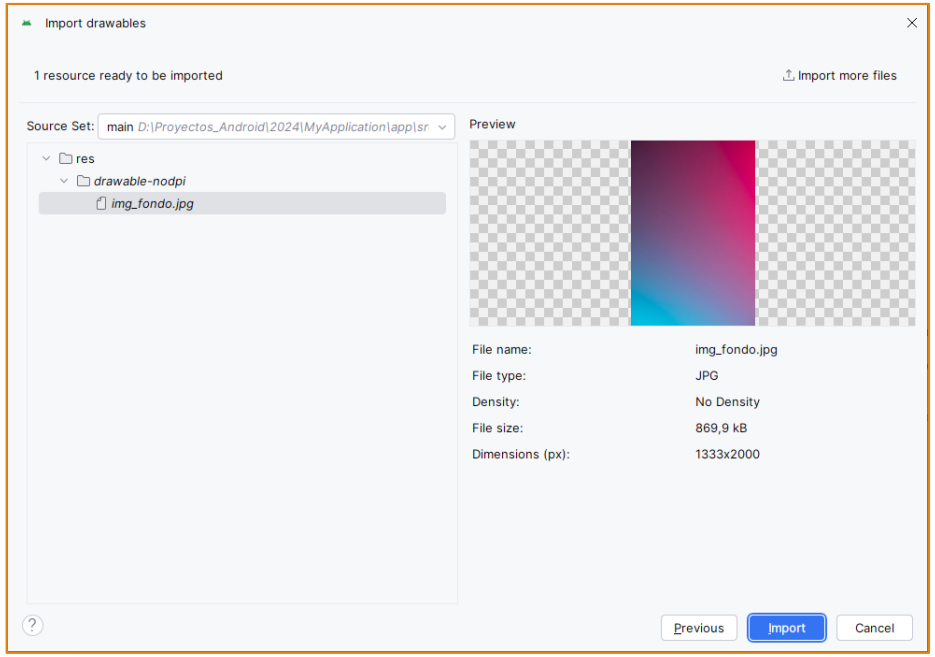
# 3. Imágenes: Importar al proyecto

A continuación buscamos la imagen y clicamos en Ok.

Una vez la hemos abierto elegimos ***Density*** en el desplegable ***QUALIFIER TYPE*** y ***No Density*** en ***VALUE***, clicamos sobre ***Next*** y por último en ***Import***.



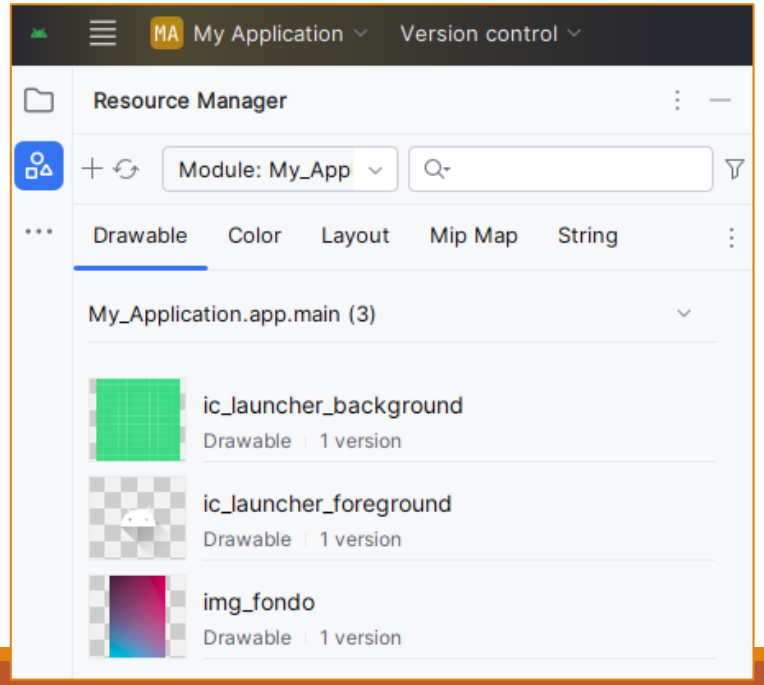



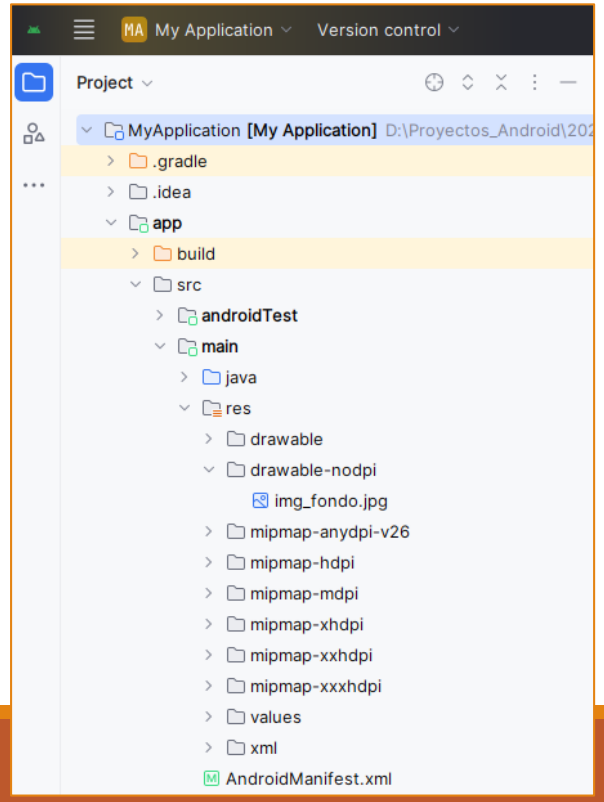



# 3. Imágenes: Importar al proyecto

Esta modificación es necesaria para que Android no escale las imágenes importadas y se puedan ver borrosas, necesiten mucha memoria o el tamaño no sea el correcto.

Esto ocurre porque los dispositivos de Android tienen diferentes tamaños de pantalla y píxeles por pulgada (ppi).



# 3. Imágenes: Código

Para añadir la imagen a nuestra aplicación hacemos uso de la función *Image* que, por defecto, acepta dos parámetros: *painter* y *contentDescription*.

- *painter*: asignamos la imagen a dibujar. Para ello antes la obtenemos usando la función *painterResource* y pasándole el nombre de la imagen importada a través de la clase *R* (*R.drawable.img_fondo*).
- *contentDescription*: Descripción del contenido de la imagen para que lectores de pantalla puedan describir la imagen. En este caso al ser un fondo decorativo que nos interesa que lo interprete lo asignamos a ***null***.

```
47  @Composable
48  fun EjemploImagen(){
49      val imagen = painterResource(id = R.drawable.img_fondo)
50
51      Image(
52          painter = imagen,
53          contentDescription = null)
54  }
```

# 3. Imágenes: Código

Algunas modificaciones que podemos hacer a nuestra imagen son:

- Escalado de la imagen: Para ello accedemos al argumento *contentScale* y elegimos el tipo de escalado que queramos.
- Opacidad de la imagen: Si queremos cambiar la opacidad de la imagen modificaremos el argumento *alpha* que toma valores desde el 0 al 1.

```
48      @Composable
49      fun EjemploImagen(){
50          val imagen = painterResource(id = R.drawable.img_fondo)
51
52          Image(
53              painter = imagen,
54              contentDescription = null,
55              contentScale = ContentScale.Crop,
56              alpha = 0.5F,
57              modifier = Modifier.fillMaxSize() // Indicamos que puede usar toda la pantalla
58          )
59      }
```

# 3. Imágenes: Código

Original





Modificada

Imagen diseñada por *Freepik*

# 4. Componente *Scaffold*

El componente *Scaffold* (andamio) de *Material Design* nos permite implementar diseños complejos de una forma sencilla.

Es el utilizado por *Android Studio* por defecto y acepta varios elementos componibles como parámetros:

- *topBar*: Es la barra de la app que se encuentra en la parte superior de la pantalla.
- *bottomBar*: Es la barra de la aplicación que se encuentra en la parte inferior de la pantalla.
- *floatingActionButton*: Un botón situado en la esquina inferior derecha de la pantalla.

```
30  class MainActivity : ComponentActivity() {
31      override fun onCreate(savedInstanceState: Bundle?) {
32          super.onCreate(savedInstanceState)
33          enableEdgeToEdge()
34          setContent {
35              MyApplicationTheme {
36                  Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
37                      Greeting(
38                          name = "Android",
39                          modifier = Modifier.padding(innerPadding)
40                      )
41                  }
42              }
43          }
44      }
45  }
```

# 4. Componente *Scaffold*

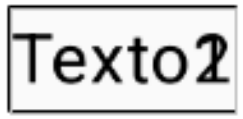
```
44              MyApplicationTheme {
45                  Scaffold(
46                      topBar = {
47                          TopAppBar(
48                              colors = topAppBarColors(
49                                  containerColor = MaterialTheme.colorScheme.primaryContainer,
50                                  titleContentColor = MaterialTheme.colorScheme.primary,
51                              ),
52                              title = {
53                                  Text( text: "Top app bar")
54                              }
55                          )
56
57                      },
58                      bottomBar = {
59                          BottomAppBar(
60                              containerColor = MaterialTheme.colorScheme.primaryContainer,
61                              contentColor = MaterialTheme.colorScheme.primary,
62                          ) {
63                              Text(
64                                  modifier = Modifier
65                                      .fillMaxWidth(),
66                                  textAlign = TextAlign.Center,
67                                  text = "Bottom app bar",
68                              )
69                          }
70                      },
71                      floatingActionButton = {
72                          FloatingActionButton(onClick = {  }) {
73
74                          }
75                      },
76                      modifier = Modifier.fillMaxSize()) { innerPadding ->
77                      Greeting(
78                          name = "Android",
79                          modifier = Modifier.padding(innerPadding)
80                      )
81                  }
82              }
```



# 5. Layouts

Una función de componibilidad puede dibujar varios elementos de la IU. Sin embargo, si no tiene indicaciones sobre cómo quieres organizarlos, *Compose* los apila uno encima del otro.

```
87  @Composable
88  fun EjemploLayout() {
89      Text( text: "Texto1")
90      Text( text: "Texto2")
91  }
```



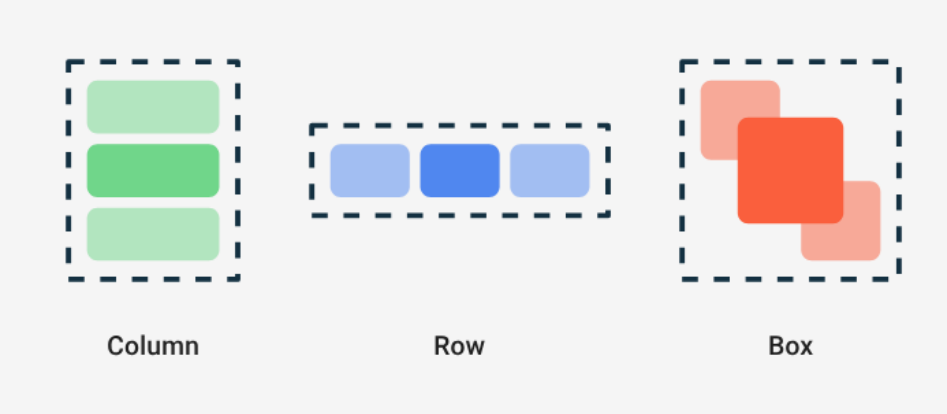

Para ello *Compose* proporciona *layouts* listos para ayudar a organizar los elementos de la IU:

- Elementos de diseño estándar
  - Column
  - Row
  - Box
- ConstraintLayout

# 5. Layouts: Elementos de diseño estándar

Los elementos de diseño estándar son componentes fundamentales que nos permiten implementar funciones que admiten composición para combinar esos diseños en uno más elaborado que se adapte a la aplicación a diseñar.

Además, *Compose* administra diseños anidados de forma eficiente, por lo que son una excelente forma de diseñar una IU compleja.

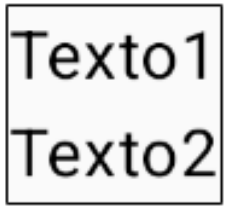



*Autor: developer.android.com*

# 5. Layouts: *Column y Row*

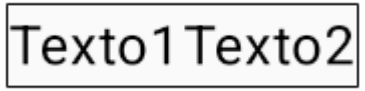
El *layout Column* nos permite posicionar los elementos verticalmente en la pantalla:

```
88      @Composable
89      fun EjemploLayout() {
90          Column {
91              Text( text: "Texto1")
92              Text( text: "Texto2")
93          }
94      }
```




El *layout Row* nos permite posicionar los elementos horizontalmente en la pantalla:

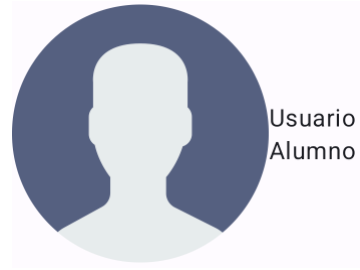
```
89      @Composable
90      fun EjemploLayout() {
91          Row {
92              Text( text: "Texto1")
93              Text( text: "Texto2")
94          }
95      }
```



# 5. Layouts: *Column y Row*

Como hemos comentado, podemos anidar los *layouts* *Row* y *Column* de una forma eficiente:

```
95  @Composable
96  fun EjemploLayout() {
97      Row(verticalAlignment = Alignment.CenterVertically) {
98          Image(painterResource(id = R.drawable.usuario), contentDescription = "Usuario")
99          Column {
100             Text( text: "Usuario")
101             Text( text: "Alumno")
102         }
103     }
```




Para posicionar los elementos dentro de un *Row* usamos los argumentos *horizontalArrangement* y *verticalAlignment*. Para un objeto *Column*, configura los argumentos *verticalArrangement* y *horizontalAlignment*:

# 5. Layouts: *Box*

El *layout* *Box* nos permite posicionar un elemento sobre otro:

```
94    @Composable
95    fun EjemploLayout() {
96        Box {
97            Image(painter = painterResource(id = R.drawable.usuario), contentDescription = "Usuario")
98            Icon(Icons.Filled.CheckCircle, contentDescription = "Check")
99        }
100   }
```

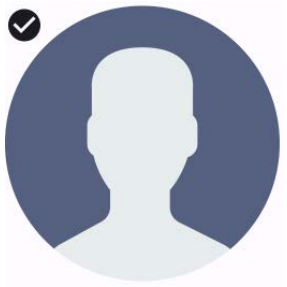


Imagen diseñada por *flaticon.com*

# 5. Layouts: Modificadores

Los modificadores te permiten decorar o modificar un elemento componible. Como por ejemplo:

- Cambiar su tamaño, diseño o comportamiento.
- Añadir información (por ejemplo, etiquetas de accesibilidad).
- Procesar las entradas del usuario.
- Añadir eventos.

Para utilizarlos haremos uso de la clase *Modifier*.

```
96      @Composable
97      fun EjemploLayout() {
98          Column (modifier = Modifier.padding(24.dp)){
99              Text( text: "Usuario")
100             Text( text: "Alumno")
101         }
```



# 5. Layouts: Modificadores

El orden al utilizar los modificadores es importante. Por ejemplo:

```
97  @Composable
98  fun EjemploLayout() {
99      Column (
100         modifier = Modifier
101             .clickable(onClick = { })
102             .padding(16.dp)
103             .fillMaxWidth()
104     ){
105         Text( text: "Usuario")
106         Text( text: "Alumno")
107     }
108 }
```

Se puede hacer clic en toda la columna (incluso en la zona del *padding*).

```
97  @Composable
98  fun EjemploLayout() {
99      Column (
100         modifier = Modifier
101             .padding(16.dp)
102             .clickable(onClick = { })
103             .fillMaxWidth()
104     ){
105         Text( text: "Usuario")
106         Text( text: "Alumno")
107     }
108 }
```

No se puede hacer clic en la zona del *padding*.

# 6. Botones

Con el elemento *Button* podemos añadir un botón a nuestra aplicación. Por defecto, usaremos el argumento *onclick* para especificar, en una función lambda, qué acción alizar cuando se pulsa el botón.

Si, por ejemplo, queremos que parezca texto en él, tendremos que utilizar el elemento *Text*.

```
98   @Composable
99   fun EjemploBoton() {
100      Column (
101          modifier = Modifier
102              .padding(16.dp)
103              .fillMaxWidth()
104      ){
105          Button(onClick = { /*TODO*/ }) {
106              Text( text: "Pulsa")
107
108          }
109      }
110  }
```



# 7. Mockups

Antes de comenzar a diseñar una aplicación es importante realizar un boceto de su interfaz de usuario (*mockup*).

Podemos hacer uso de las aplicaciones existentes para crear *mockups* o simplemente crearlos dibujando en un papel.

En ellos especificaremos dónde se posicionarán los diferentes elementos que forman la interfaz de usuario de la aplicación.

Algunas aplicaciones para diseñarlos son:

- https://moqups.com
- https://www.mockflow.com

# 8. Localización

Cuando **subimos** una aplicación a ***Google Play*** tenemos un amplio rango de usuarios. Es difícil anticipar qué tipo de dispositivo se usará (teléfono, tablet, una versión antigua de *Android* o diferentes idiomas de los que usamos).

Hay muchas **buenas prácticas** que podemos utilizar a la hora de crear una aplicación para hacer que trabajen bien para este tipo de usuarios.

Una de ellas es **localizar** nuestra aplicación, hay una lista de tareas para la localización que podemos usar:

https://developer.android.com/distribute/best-practices/launch/localization-checklist

# 8. Localización

Localización significa adaptar nuestra aplicación a diferentes sitios. En el sentido de combinaciones de idioma y país para hacer una positiva experiencia a todos los tipos de usuarios.

Por ahora nos vamos a enfocar en cómo hacer que nuestra aplicación soporte diferentes idiomas.

Podemos ver que desde el móvil *Android* se puede elegir el idioma a utilizar de todos los que soporta *Android*. Si cambiamos el idioma (por ejemplo inglés) podemos ver que todo se traduce a ese idioma. Además, las aplicaciones que soportan ese idioma aparecerán traducidas.

Aquí podemos ver cómo dar soporte a diferentes idiomas:

https://developer.android.com/training/basics/supporting-devices/languages

https://developer.android.com/guide/topics/resources/localization

# 8. Localización

Dentro del directorio **res** puedes crear subdirectorios que contengan **recursos** para un determinado **idioma**.

```
MyProject/
    res/
        values/
            strings.xml
        values-es/
            strings.xml
        values-fr/
            strings.xml
```

***values*** es el directorio por defecto y ahí estarán todos los *strings* en **inglés**. En **values-es** estarán los *string* en **español** y en **values-fr** estarán en **francés**.

Si el usuario elige el **español** en su dispositivo se cargarán los *strings* de la carpeta **values-es**. Si la carpeta no existe cogerá los *strings* de la carpeta por defecto (**values**).

Se puede ver que **es** es el código para español y **fr** para francés. Android sigue los **códigos** del **ISO 639-1**: https://es.wikipedia.org/wiki/ISO_639-1

# 8. Localización

Para que el sistema pueda traducir los textos en nuestra aplicación, necesitamos aislar el texto de toda la demás lógica de la aplicación. Esto lo hacemos creando un fichero ***strings.xml*** en la carpeta ***values***.

Este fichero contiene una lista de todos los recursos *string* que son visibles al usuario.

Inglés (configuración regional predeterminada), `/values/strings.xml`:

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="title">My Application</string>
    <string name="hello_world">Hello World!</string>
</resources>
```

Español, `/values-es/strings.xml`:

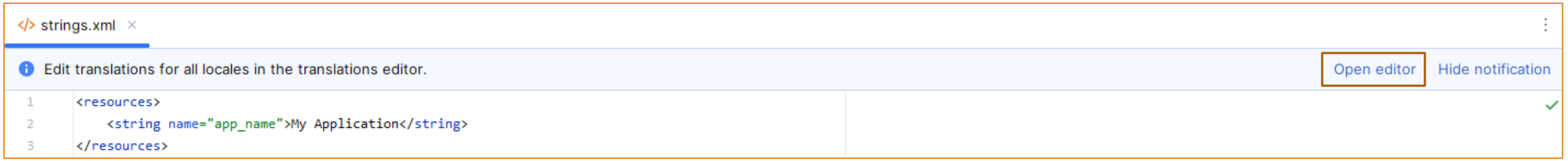
```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="title">Mi Aplicación</string>
    <string name="hello_world">Hola Mundo!</string>
</resources>
```

# 8. Localización

Ahora vamos a proveer de traducciones alternativas en otro idioma.

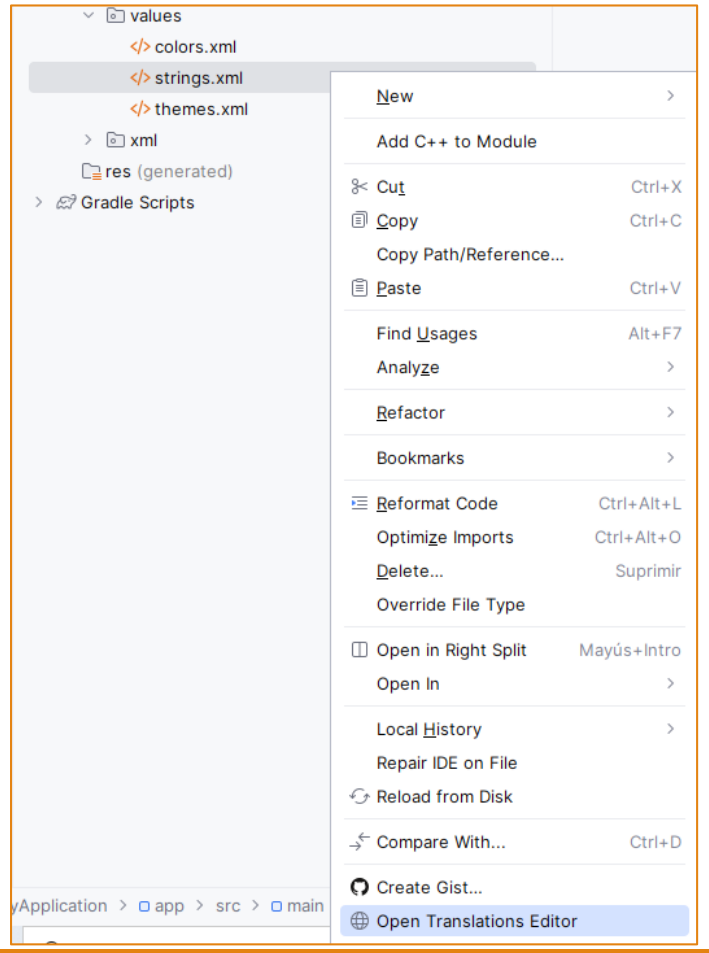
Para eso en *Android Studio* hay una herramienta llamada **editor de traducciones**.

Para abrirlo podemos clicar en **Open editor**



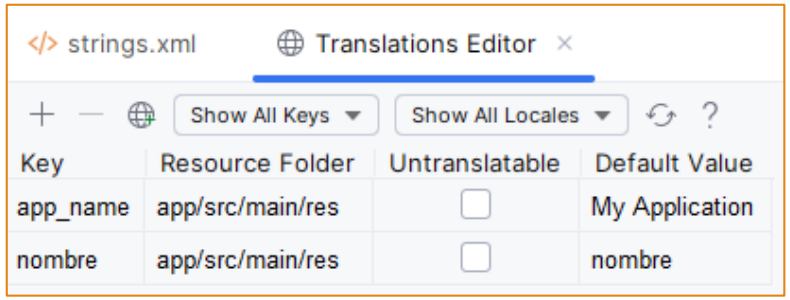
# 8. Localización

También podemos acceder a él clicando con el botón derecho sobre el fichero ***strings.xml*** y después en ***Open Translations Editor***

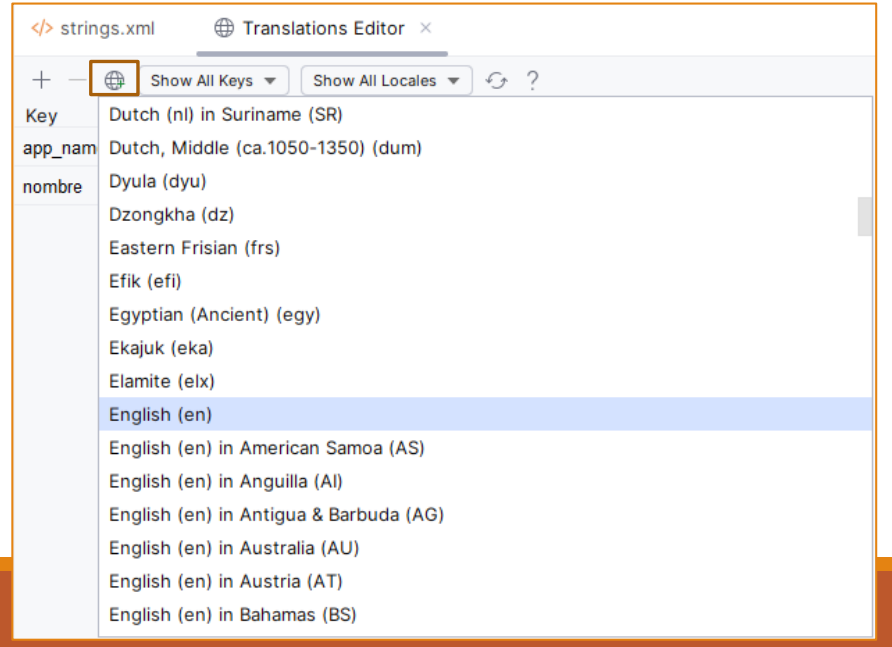


# 8. Localización

Podemos ver el nombre y el valor por defecto de las *strings* utilizadas.

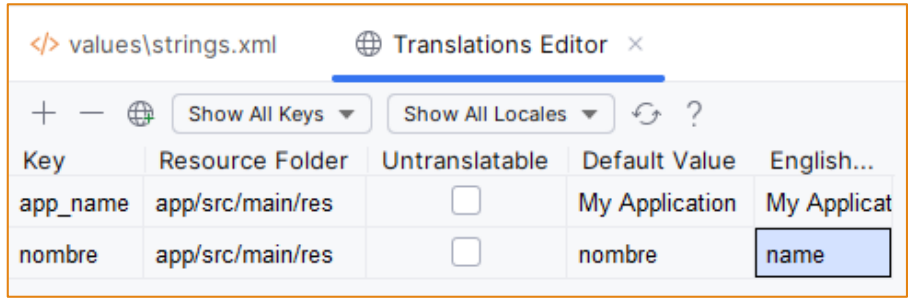



Si pulsamos sobre el globo podemos introducir nuevos idiomas.



# 8. Localización

De esta forma podremos traducir todos los elementos de la aplicación o pulsar sobre la opción ***Untranslatable*** para evitar su traducción.




Además podremos ver que se creará un fichero *strings.xml* por cada idioma utilizado.

